TECHNICAL SPECIFICATION & ARCHITECTURAL OVERVIEW

KnowledgeTrace MCQ: Full Project Architecture & System Design

A Multi-Tiered Cognitive Diagnostic Assessment & Bayesian Knowledge Tracing Platform for Multiple-Choice Questions

Author / System: KnowledgeTrace Core Team	**Date / Release:** 2026 Production Build	**Architecture:** 3-Layer Cognitive Engine
Backend Stack: FastAPI + Async SQLAlchemy	**Frontend Stack:** React (Vite) + Tailwind CSS	**Database:** PostgreSQL (Supabase Cloud)

1. Executive Overview & Purpose of the Project

Standard educational assessment platforms evaluate student performance using naive percentage scores (e.g., 7/10 correct = 70%). This approach has profound limitations: it fails to reveal *why* a student missed a question, whether the failure stems from a missing foundational prerequisite or a careless slip, and how knowledge evolves dynamically over time. Multiple-choice tests in particular suffer from guessing distortions (a 25% chance of being right by random chance on a 4-option item) and slipping (making an arithmetic or reading error despite knowing the core concept).

KnowledgeTrace MCQ was engineered from the ground up to solve this fundamental problem. It is a comprehensive, production-ready learning platform that combines **Bayesian Knowledge Tracing (BKT)**, **Cognitive Diagnostic Models (DINA)**, and **Large Language Model (LLM) Concept Graphs** to deliver fine-grained, actionable diagnostic reports.

CORE MISSION & OBJECTIVES

Primary Engineering Objectives:

- **Prerequisite Graph Inference:** Automatically infer directed curriculum dependencies and validate them as Directed Acyclic Graphs (DAGs) to prevent circular prerequisites.
- **Dynamic Mastery Tracking:** Update student concept mastery probabilities in real-time as each question is answered using Bayesian forward updating.
- **Multi-Skill Cognitive Diagnosis:** Map complex questions to multiple skills via a Q-Matrix, and use exact Bayesian marginalization to diagnose root-cause skill deficits.
- **Actionable Student & Teacher Dashboards:** Deliver instantaneous feedback identifying exactly which prerequisite skills are missing and provide teachers with class-wide gap analytics.

2. Complete Technology Stack

The application is built on an enterprise-grade asynchronous architecture designed for high throughput, numerical efficiency, and modular extensibility:

Layer / Subsystem	Technologies Used	Role & Architectural Responsibility
Backend Core	Python 3.10+, FastAPI, Uvicorn (ASGI)	High-performance asynchronous REST API handling auth, test delivery, and diagnostic dispatch.
Database & ORM	PostgreSQL (Supabase), SQLAlchemy 2.0 Async, asyncpg	Connection-pooled async database engine with full relational schemas, constraints, and indexes.
Security & Auth	JWT (python-jose), Passlib (Bcrypt), OAuth2 Bearer	Stateless token-based authentication with role-based access control (Teacher vs Student).

Cognitive & ML Engine	NumPy, NetworkX, Groq API (groq/compound), pyBKT	Vectorized Bayesian DINA marginalization, DAG cycle detection, and LLM-based skill extraction.
Frontend UI/UX	React 18, Vite, Tailwind CSS, Lucide React	Modern glassmorphism interface, real-time response telemetry, and animated diagnostic graphs.
DevOps & Deployment	Docker, Docker Compose, Nginx (Alpine), Multi-stage builds	Isolated container network (mcq-net), production Nginx reverse proxy, and zero-downtime execution.

3. End-to-End Data Pipeline & System Flow

Data travels through a multi-stage lifecycle connecting teachers, LLM reasoning services, student interactions, and psychometric diagnosis algorithms:

• **Step 1 — Curriculum Authoring:** The teacher creates knowledge concepts (e.g., 'Linear Algebra', 'Eigenvalues', 'Matrix Multiplication') via the Teacher Dashboard or API.

• **Step 2 — Automated Prerequisite Graph Construction:** A batch request is sent to the Groq LLM API. The model infers prerequisite dependencies with pedagogical reasoning and confidence scores. NetworkX constructs an in-memory graph, validates Directed Acyclic Graph (DAG) invariants, rejects cycle-causing edges, and writes validated edges to concept_edges.

• **Step 3 — Test Creation & Bulk Parsing:** The teacher authors tests either through an interactive UI or via the Bulk Question Parser, which uses a deterministic regex state machine to parse markdown text formatted as Q1 — Concept: ... A) ... B) ... Answer: ....

• **Step 4 — Automated Q-Matrix Attribution:** When questions are saved, an asynchronous background task calls the LLM service to analyze question text and distractor options, identifying up to 2 additional required prerequisite skills (thresholded at confidence ≥ 0.60) and writing them into the question_skills multi-attribute matrix.

• **Step 5 — Test Assignment & Delivery:** Teachers assign tests to whole cohorts or individual students by email. Students receive the assignment on their personal dashboard.

• **Step 6 — Test Taking & Real-time Telemetry:** As the student navigates questions, a high-resolution millisecond timer logs exact response latency (response_time_ms), the selected option, and correctness outcome. The response is posted to POST /student/responses.

• **Step 7 — Asynchronous Diagnostic Pipeline:** The API immediately commits the student response to Postgres and dispatches an asynchronous background task executing two coordinated analytical layers:
(a) Layer 2 (BKT): Updates the student's chronological mastery probability $P(L_t)$ for the primary concept using Bayesian forward filtering.
(b) Layer 3 (DINA): Identifies all interconnected skills via question skill co-occurrence (BFS connected component graph partition). Marginalizes over the exact 2^K joint probability space using log-sum-exp normalization to produce posterior mastery probabilities, confidence metrics ($1 - 4p(1-p)$), and reliability tiers (LOW, MEDIUM, HIGH).

• **Step 8 — Instant Diagnostic Feedback:** Students inspect the Diagnostic Report to see exact skill gaps and question-level 'WHY' explanations showing which missing sub-skill caused each mistake. Teachers view class-level aggregated pass rates and item difficulty statistics.

PERFORMANCE & CONCURRENCY ARCHITECTURE

Decoupled Asynchronous Execution:
Because multi-attribute DINA Bayesian marginalization involves combinatorial joint state spaces (2^K) and database lookups, it runs entirely within FastAPI BackgroundTasks. The student receives sub-50ms HTTP response confirmation for their answer submission while the heavy psychometric math executes asynchronously in the background.

4. Backend Architecture & Implementation Structure

The backend repository is organized according to clean separation of concerns: core configurations, database sessions, domain routers, Pydantic validation schemas, and psychometric domain services:

```
backend/
■■■ app/
■ ■■■ main.py # Application factory, CORS, lifespan migration, router mounting
■ ■■■ database.py # Async engine, async_sessionmaker, get_db dependency
■ ■■■ models.py # SQLAlchemy ORM declarations (11 tables)
■ ■■■ auth.py # JWT token generation, bcrypt password hashing, auth dependencies
■ ■■■ core/
■ ■ ■■■ config.py # Pydantic BaseSettings loading .env, database URLs, Groq keys
■ ■■■ schemas/ # Pydantic v2 validation contracts
■ ■ ■■■ user.py # UserCreate, UserRead, Token, TokenData
■ ■ ■■■ test.py # TestCreate, TestRead, TestAssignmentCreate
■ ■ ■■■ question.py # QuestionCreate, OptionCreate, QuestionStudentRead
■ ■ ■■■ response.py # ResponseCreate, ResponseRead
■ ■ ■■■ concept.py # ConceptCreate, ConceptRead, ConceptEdgeRead
■ ■ ■■■ diagnosis.py # CognitiveProfileResponse, QuestionWhyResponse
■ ■ ■■■ bulk_import.py # BulkQuestionImportRequest, BulkQuestionImportResponse
■ ■■■ routers/ # Domain API endpoints
■ ■ ■■■ auth.py # /auth/register, /auth/login
■ ■ ■■■ teacher.py # /teacher/tests, /teacher/concepts, /teacher/assign-test, bulk parsing
■ ■ ■■■ student.py # /student/assignments, /student/responses, test completion
■ ■ ■■■ concepts.py # /concepts/infer-graph, /concepts/graph
■ ■ ■■■ mastery.py # /mastery/student/{id}, /mastery/recompute/{id}
■ ■ ■■■ diagnosis.py # /diagnosis/student/{id}, /diagnosis/.../why, /report
■ ■■■ services/ # Pure domain logic & ML algorithms
■ ■■■ concept_graph.py # LLM prerequisite inference & NetworkX DAG cycle prevention
■ ■■■ llm_service.py # Groq API client with structured JSON completion & prompt templates
■ ■■■ mastery_service.py # Abstract BaseMasteryEstimator & PYBKTMasteryEstimator (BKT)
■ ■■■ dina_service.py # Multi-Attribute Bayesian DINA diagnosis & graph block partition
■■■ Dockerfile # Production container definition (python:3.11-slim)
■■■ requirements.txt # Pinned production dependencies
```

Key Backend Endpoints Summary

Endpoint	Method	Auth / Role	Description
/auth/register	POST	Public	Registers student or teacher account with hashed password.
/auth/login	POST	Public	Authenticates credentials; returns signed JWT with user role.
/teacher/tests	GET/POST	Teacher	Creates or lists tests with questions and answer options.
/teacher/assign-test	POST	Teacher	Assigns tests to students by user IDs or email list.
/teacher/tests/parse-bulk-questions	POST	Teacher	State-machine regex parser converting formatted text into questions.
/concepts/infer-graph	POST	Teacher	Triggers LLM prerequisite extraction with NetworkX DAG validation.
/student/assignments	GET	Student	Retrieves all assigned tests and their completion statuses.
/student/responses	POST	Student	Submits answer; dispatches background BKT and DINA computations.
/diagnosis/student/{id}/report	GET	Authenticated	Generates question-level diagnostic breakdown with root-cause skill deficits.
/diagnosis/student/{id}	GET	Authenticated	Retrieves full multi-skill cognitive mastery inventory and confidence scores.

5. Frontend Architecture & User Interface Design

The frontend is engineered as a responsive Single Page Application (SPA) using React 18, Vite, and Tailwind CSS. It features a sleek glassmorphism visual language, custom color tokens, and role-based client-side routing:

- **Centralized Authentication Context (AuthContext.jsx):** Stores user metadata, token claims, and session status in localStorage. An Axios interceptor automatically injects the Authorization: Bearer <token> header on every outgoing API request.
- **Role-Guarded Navigation (ProtectedRoute.jsx):** Prevents unauthorized access across routes. Teachers attempting to access student views or students attempting to access teacher views are instantly redirected to their respective dashboards.
- **Teacher Dashboard (TeacherDashboard.jsx):** Displays all authored assessments, participant counts, and aggregate results. Contains an assignment modal supporting both multi-select student checkboxes and instant email enrollment.
- **Dual-Mode Test Builder (TestBuilder.jsx):** Allows teachers to build tests interactively question-by-question or paste an entire test script into the Bulk Import Parser, which verifies syntax and matches questions to existing database concepts in real time.
- **Student Examination Environment (TakeTest.jsx):** A focused, distraction-free testing interface. Records millisecond latency using useRef timestamp differentials, preventing timing drift and providing exact behavioral telemetry.
- **Cognitive Diagnostic Portal (DiagnosticReport.jsx):** Visualizes the student's mastery profile. Presents top-line metric cards (Overall Mastery, Identified Gaps, Analyzed Questions), question-level missing prerequisite cards tagged with *'ANALYSED VIA DINA'*, and a skill inventory bar chart color-coded by mastery tiers.

6. Database Design & Relational Schema

The relational database schema is deployed on PostgreSQL via Supabase. It comprises 11 tightly coordinated tables enforcing strict referential integrity, foreign key cascades, unique constraints, and optimized index lookups:

Table Name	Primary Key & Foreign Keys	Core Attributes & Types	Pedagogical Function & Role

app_users	PK: id	name (str), email (unique), hashed_password, role ('teacher'/'student')	Central identity table storing credentials and system privileges.
concepts	PK: id FK: created_by_teacher_id	name (unique, str)	Curriculum knowledge units/topics that tests and questions assess.
concept_edges	PK: id FK: from_concept_id FK: to_concept_id	confidence (float, default 1.0), source ('llm_inferred'/'manual')	Directed prerequisite graph edges validated as a DAG by NetworkX.
tests	PK: id FK: created_by_teacher_id	title (str), created_at (timestamp)	Assessment containers created by teachers to bundle questions.
questions	PK: id FK: test_id FK: concept_id	text (str), correct_option_id (nullable int)	Individual MCQ items mapped to a primary assessed concept.
options	PK: id FK: question_id	text (str)	Multiple choice alternatives presented to students for each question.
test_assignments	PK: id FK: test_id FK: student_id	status ('not_started'/'in_progress'/'completed')	Tracks student enrollment and execution status for assigned tests.
responses	PK: id FK: student_id FK: question_id FK: selected_option_id	is_correct (bool), response_time_ms (int), answered_at (timestamp)	Granular interaction logs capturing correctness and millisecond latency.
student_concept_mastery	PK: id FK: student_id FK: concept_id	mastery_p (float [0, 1]), opportunity_count (int), last_updated	Layer 2 Bayesian Knowledge Tracing (BKT) sequential mastery estimates.
question_skills	PK: id FK: question_id FK: concept_id **UQ:** (question_id, concept_id)	is_primary (bool), source ('teacher'/'llm_inferred'), llm_confidence (float)	Multi-attribute Q-Matrix mapping items to all required skills for DINA.
cognitive_diagnosis_profile	PK: id FK: student_id FK: concept_id **UQ:** (student_id, concept_id)	raw_mastery_p, adjusted_mastery, confidence_score, reliability_tier	Layer 3 Bayesian DINA posterior cognitive diagnosis profiles.

DATABASE PERFORMANCE & INTEGRITY GUARANTEES

Database Optimization Strategy:
1. **Composite Unique Constraints:** (question_id, concept_id) in question_skills and (student_id, concept_id) in cognitive_diagnosis_profile enable atomic upserts via PostgreSQL ON CONFLICT DO UPDATE, preventing race conditions during concurrent test submissions.
2. **Indexed Foreign Keys:** High-frequency queries (e.g., fetching a student's entire diagnostic profile or all skills for a question) execute with $O(1)$ index seeks via dedicated B-tree indexes on student_id and concept_id.